

Guía de Consulta Práctica

Trabajo Práctico 04 - Parte C - Estimación de Costos

1.- Estimación de Costos en Proyectos de Software

La estimación de costos es una herramienta clave para la toma de decisiones informadas en proyectos de desarrollo de software. Su objetivo no es solo calcular cuánto costará un sistema, sino también anticipar el esfuerzo requerido, evaluar la relación costo-beneficio, y establecer condiciones de viabilidad financiera y operativa.

Una estimación bien formulada permite:

- Definir cronogramas realistas.
- Asignar recursos de forma eficiente.
- Justificar inversiones ante actores institucionales o comunitarios.
- Identificar riesgos financieros y operativos.
- Comparar alternativas tecnológicas o metodológicas.

1.1. Tipos de Estimación según el momento del Proyecto

Tipo de Estimación	Etapas del Proyecto	Nivel de Precisión	Finalidad Principal
Preliminar	Inicio o idea	Baja	Viabilidad general, comparación de alternativas
Intermedia	Diseño funcional	Media	Planificación de recursos, análisis de riesgos
Detallada	Desarrollo avanzado	Alta	Control de ejecución, ajustes presupuestarios

1.2. Factores que influyen en la Estimación

- **Tamaño Funcional del Sistema** (cantidad de módulos, casos de uso, pantallas, reportes).
- **Complejidad Técnica** (interoperabilidad, seguridad, escalabilidad).
- **Nivel de Experiencia del Equipo** (curva de aprendizaje, productividad esperada).
- **Condiciones del Entorno** (infraestructura disponible, conectividad, soporte institucional).
- **Modelo de Desarrollo** (ágil, en cascada, incremental, híbrido).

2. Métodos de Estimación de Costos en Proyectos de Software

La estimación de costos es una actividad crítica en las etapas iniciales de un proyecto. No solo orienta la planificación financiera, sino que también permite anticipar riesgos, justificar decisiones y evaluar la sostenibilidad del sistema propuesto. Existen diversos métodos, cada uno con fortalezas y limitaciones según el tipo de proyecto, el nivel de información disponible y el grado de formalidad requerido.

2.1. Métodos principales

Método	Descripción Breve	Ventajas	Limitaciones	Contexto Ideal
COCOMO II	Modelo matemático basado en líneas de código o puntos de función.	Formal, ajustable, detallado	Requiere datos técnicos precisos	Proyectos grandes y complejos
Puntos de Función	Mide la funcionalidad desde la perspectiva del usuario.	Independiente de tecnología	Subjetivo, depende del analista	Sistemas con fuerte interacción funcional
Analogía	Compara con proyectos similares ya realizados.	Rápido, simple	Menos preciso, depende de antecedentes	Proyectos pequeños o estimaciones iniciales
Estimación por Componentes	Descompone el sistema en módulos funcionales y estima cada uno por separado.	Flexible, permite ajustes parciales	Requiere buena definición modular	Sistemas con arquitectura distribuida o modular

2.2. Métodos Complementarios y Combinados

En contextos críticos, comunitarios o con alta sensibilidad operativa, puede ser útil **combinar métodos** para mejorar la precisión, la trazabilidad y la validación cruzada. Estas combinaciones permiten compensar las limitaciones de cada enfoque y adaptarse mejor a la realidad del entorno.

- **COCOMO II + Analogía.** Se utiliza para validar estimaciones en proyectos grandes, comparando los resultados matemáticos con antecedentes similares. Esto permite ajustar coeficientes y detectar desviaciones tempranas.
- **Puntos de Función + Consulta a Actores Clave.** Ideal para estimar funcionalidad desde la perspectiva del usuario, incorporando la experiencia situada de coordinadores, técnicos y usuarios finales. Mejora la pertinencia y la aceptabilidad del sistema.
- **Estimación por Componentes + Simulación Operativa.** Permite descomponer el sistema en partes funcionales y observar tiempos reales de implementación en pruebas piloto o simulaciones. Es útil para ajustar cronogramas y prever sobrecargas operativas.

2.3. Buenas Prácticas para la Estimación

- **Documentar Supuestos:** registrar qué se considera incluido o excluido en la estimación.
- **Incluir Costos Ocultos:** como capacitación, soporte, conectividad, mantenimiento.
- **Validar con Expertos:** contrastar la estimación con desarrolladores, coordinadores y usuarios.
- **Revisar Periódicamente:** ajustar la estimación según avances, cambios de alcance o condiciones externas.
- **Alinear con Objetivos Específicos:** vincular cada componente estimado con los objetivos funcionales del sistema.

3. Estimación de Líneas de Código (LOC) a partir de Módulos Funcionales

3.1. ¿Por qué partir de los Módulos?

Los módulos funcionales representan unidades operativas del sistema que responden a objetivos específicos. Estimar LOC por módulo permite:

- Descomponer el esfuerzo en partes manejables.
- Asignar recursos por componente.
- Ajustar estimaciones según la complejidad de cada módulo.
- Facilitar la validación cruzada con casos de uso y requerimientos.

3.2. Pasos para estimar LOC por Módulo

1. Identificar los Módulos Funcionales del Sistema. Ejemplo: Registro de usuarios, gestión de pagos, generación de reportes, administración de roles, etc.

2. Asociar cada Módulo con sus Funcionalidades Clave. Ejemplo:

- **Módulo “Revisión por Áreas Evaluadoras”.** asignar expediente a Secretaría Académica, registrar observaciones, transferir expediente al Departamento de Alumnos, devolver expediente por inconsistencias, cerrar revisión en Comisión de Ingreso.

3. Estimar LOC por Funcionalidad La estimación de LOC por módulo se basa en descomponer cada componente del sistema en sus funcionalidades clave y asignar un rango de líneas de código esperadas según la complejidad de cada funcionalidad. Este enfoque permite construir estimaciones progresivas, trazables y ajustables.

3.3. Criterios utilizados para estimar LOC

Para cada funcionalidad se considera:

- **Tipo de Operación:** alta, edición, consulta, validación, exportación, etc.
- **Nivel de Complejidad:** lógica simple, validaciones cruzadas, interacción con bases de datos, generación de documentos.
- **Interfaz Esperada:** formularios, filtros, dashboards, reportes.
- **Integración Externa:** si requiere conexión con APIs, servicios o sistemas externos.

Se asigna un rango referencial de LOC por tipo de funcionalidad:

Tipo de funcionalidad	LOC estimado (referencial)
Alta/edición simple	100–300 LOC
Validación con reglas complejas	200–400 LOC
Consulta con filtros	200–500 LOC
Exportación de reportes	400–800 LOC
Dashboard con indicadores	500–900 LOC

Sumar el Total por Módulo Ejemplo aplicado:

Módulo Funcional	Funcionalidades Clave	Estimación por Funcionalidad	LOC Estimado
Registro de aspirantes	Alta de datos personales, edición de datos, validación de campos obligatorios	Alta (200) + Edición (200) + Validación (200)	600 LOC
Carga de documentación	Subida de archivos, verificación de formato, asociación al expediente	Subida (300) + Verificación (300) + Asociación (300)	900 LOC
Revisión por áreas evaluadoras	Asignación de expediente, registro de observaciones, transferencia entre etapas	Asignación (300) + Observaciones (400) + Transferencia (300)	1.000 LOC
Seguimiento de expediente	Consulta de estado, notificaciones, historial de revisiones	Consulta (300) + Notificación (300) + Historial (300)	900 LOC
Reportes operativos	Listados por etapa, filtros por estado, exportación PDF/Excel	Listado (300) + Filtros (300) + Exportación (400)	1.000 LOC
Gestión de observaciones	Registro de devoluciones, motivos, acciones requeridas	Registro (300) + Motivos (200) + Acciones (300)	800 LOC
Administración de usuarios	Alta de perfiles, edición, asignación de permisos por área	Alta (200) + Edición (200) + Asignación (400)	800 LOC

Total Estimado Parcial = 6000 LOC → 6 KLOC

3.4. Estrategias para ajustar la Estimación

- **Complejidad Esperada:** aumentar LOC si hay lógica compleja, validaciones cruzadas o integración con sistemas externos.
- **Nivel de Reutilización:** reducir LOC si se usan componentes ya desarrollados o frameworks con funcionalidades integradas.
- **Lenguaje de Programación:** adaptar la estimación según el lenguaje (por ejemplo, Python suele requerir menos LOC que Java para tareas similares).

3.5. Recomendaciones para aplicar el Método

- Descomponer cada módulo en funcionalidades concretas.
- Asignar LOC por funcionalidad según complejidad estimada.
- Validar estimaciones con antecedentes o prototipos.
- Ajustar según el lenguaje de programación y el nivel de reutilización.
- Documentar los criterios usados para facilitar la revisión y trazabilidad.

🎯 4. Puntos de Función

🔍 4.1. ¿Qué son los Puntos de Función?

Los puntos de función son una medida del **tamaño funcional** de un sistema de software, basada en lo que el sistema **hace desde la perspectiva del usuario**. Evalúan la cantidad y complejidad de las funciones que el sistema debe realizar, sin depender del lenguaje de programación ni de la tecnología utilizada.

Este enfoque es especialmente útil en etapas tempranas del proyecto, cuando aún no se ha definido la arquitectura técnica, pero sí se conocen los requerimientos funcionales.

💡 4.2. ¿Cómo se calculan?

Se identifican cinco tipos de componentes funcionales:

Tipo de Componente	Descripción Breve	Ejemplo en Las Colinas
EE – Entrada Externa	Datos ingresados por el usuario que modifican el sistema	Carga de datos personales y académicos por parte del aspirante
SE – Salida Externa	Información generada por el sistema hacia el usuario	Notificación del estado del expediente y observaciones registradas
CE – Consulta Externa	Solicitud de información sin modificar datos	Consulta del historial de revisión del expediente por parte del aspirante
AI – Archivo Interno	Base de datos mantenida por el sistema	Base de datos de expedientes digitales y registros de revisión
AE – Archivo Externo	Datos utilizados pero mantenidos por otro sistema	Validación de certificados mediante sistemas externos del Ministerio de Educación

Cada componente se clasifica por **complejidad** (baja, media, alta) y se le asigna un valor estándar:

Tipo de Componente	Baja	Media	Alta
Entrada Externa	3	4	6
Salida Externa	4	5	7
Consulta Externa	3	4	6
Archivo Interno	7	10	15
Archivo Externo	5	7	10

4.3. Ejemplo aplicado: Sistema de Gestión de Admisiones – Universidad Nacional de Las Colinas

Supongamos que los estudiantes deben estimar el tamaño funcional de una aplicación para gestionar el proceso de admisión de aspirantes en la Universidad Nacional de Las Colinas. Para ello, se identifican los componentes funcionales observables por el usuario, basados en los requerimientos, casos de uso y funcionalidades descritas en el escenario.

◆ Entradas Externas (EE) – 7

¿Qué se considera una Entrada Externa? Toda operación en la que el usuario ingresa datos que modifican el estado del sistema. **Ejemplos en Las Colinas:**

- Carga de datos personales por parte del aspirante.
- Carga de datos académicos.
- Subida de certificados analíticos.
- Subida de DNI escaneado.
- Subida de fotos carnet.
- Registro de recepción física en oficina de admisiones.
- Registro de observaciones por parte de áreas evaluadoras.

¿Cómo se determina la cantidad? Se analiza cada formulario o pantalla que implique ingreso de datos con validación y almacenamiento. Se cuentan como entradas separadas si afectan entidades distintas o tienen lógica propia.

◆ Salidas Externas (SE) – 5

¿Qué se considera una Salida Externa? Información generada por el sistema que se presenta al usuario, especialmente si incluye cálculos, formatos o transformaciones. **Ejemplos:**

- Notificación del estado del expediente.
- Reporte de observaciones por área.
- Comprobante de recepción de documentación.
- Estadísticas de avance por etapa.
- Informe final de admisión por aspirante.

¿Cómo se determina la cantidad? Se identifican los reportes y visualizaciones generadas automáticamente. Se cuentan por separado si tienen estructuras, filtros o destinatarios distintos.

◆ Consultas Externas (CE) – 4

¿Qué se considera una Consulta Externa? Operaciones en las que el usuario accede a información sin modificarla. **Ejemplos:**

- Consulta del historial de revisión del expediente.
- Visualización de requisitos pendientes.
- Consulta de fechas estimadas por etapa.
- Consulta de observaciones registradas.

¿Cómo se determina la cantidad? Se analizan los casos de uso que permiten recuperar información. Se cuentan por separado si implican filtros o vistas distintas.

◆ Archivos Internos (AI) – 3

¿Qué se considera un Archivo Interno? Bases de datos mantenidas por el sistema. **Ejemplos:**

- Base de datos de aspirantes.
- Registro de expedientes digitales.
- Historial de revisiones y observaciones.

¿Cómo se determina la cantidad? Se identifican las entidades principales que el sistema gestiona de forma autónoma.

◆ Archivos Externos (AE) – 1

¿Qué se considera un Archivo Externo? Datos utilizados por el Sistema, pero mantenidos por otro sistema. **Ejemplo:**

- Validación de certificados mediante sistemas externos del Ministerio de Educación.

¿Cómo se determina la cantidad? Se revisan las integraciones previstas con sistemas externos.

4.4. Cálculo

Componente Funcional	Cantidad	Complejidad	Cálculo
Entradas Externas (EE)	7	Media	$7 \times 4 = 28$
Salidas Externas (SE)	5	Alta	$5 \times 7 = 35$
Consultas Externas (CE)	4	Baja	$4 \times 3 = 12$
Archivos Internos (AI)	3	Media	$3 \times 10 = 30$
Archivos Externos (AE)	1	Alta	$1 \times 7 = 7$

Total Estimado = 28 + 35 + 12 + 30 + 7 = 112 Puntos de Función

4.5. Conversión a LOC (opcional)

Lenguaje	Coeficiente	LOC estimado
Java	50 LOC/PF	$112 \times 50 = 5.600 \text{ LOC}$
Python	40 LOC/PF	$112 \times 40 = 4.480 \text{ LOC}$
C++	60 LOC/PF	$112 \times 60 = 6.720 \text{ LOC}$

5. COCOMO II

5.1. ¿Qué es COCOMO II?

COCOMO II (Constructive Cost Model II) es un modelo de estimación de costos y esfuerzo para proyectos de software, desarrollado por Barry Boehm. Utiliza fórmulas matemáticas basadas en el tamaño del sistema (medido en líneas de código o puntos de función) y ajustadas por múltiples factores de escala y esfuerzo, como:

- Fiabilidad requerida
- Complejidad del producto
- Experiencia del equipo
- Restricciones de tiempo
- Herramientas utilizadas

El modelo permite calcular:

- **Esfuerzo estimado** (en persona-mes)
- **Duración del proyecto**
- **Cantidad de personal necesario**
- **Costos aproximados**

5.2. Estructura del modelo

COCOMO II tiene tres niveles de aplicación:

Nivel	Descripción	Uso recomendado
Aplicación inicial (Early Design)	Estimación rápida con pocos datos, usando características generales del sistema.	Etapas tempranas, anteproyectos.
Post-architecture	Estimación detallada con información sobre arquitectura, módulos y procesos.	Proyectos con diseño definido.
Reutilización	Estima el esfuerzo para adaptar componentes existentes.	Sistemas que integran software heredado.

5.3. Fórmula base

La fórmula principal del modelo es:

$$\text{Esfuerzo (persona-mes)} = A \times (\text{Tamaño})^B \times \text{EAF}$$

Donde:

- **A**: constante empírica (por defecto, 2.94)
- **Tamaño**: en miles de líneas de código (KLOC) o puntos de función
- **B**: factor de escala (depende de características del proyecto)
- **EAF**: multiplicador de esfuerzo (Effort Adjustment Factor), calculado según 17 atributos del proyecto

5.3.1 ¿Cómo se determina el factor B?

El valor de **B** se calcula a partir de cinco **factores de escala**, cada uno evaluado en una escala cualitativa (muy bajo, bajo, nominal, alto, muy alto, extra alto). Cada nivel tiene un **valor numérico** asociado. La fórmula para B es:

$$B = 0.91 + 0.01 \times \Sigma(\text{SFi})$$

Donde **SFi** son los valores numéricos asignados a los cinco factores de escala.

Los 5 factores de escala

Factor de escala	Descripción	Ejemplo aplicado a Las Colinas
PREC (Precedencia)	Nivel de madurez del proceso	Bajo: proceso de admisión con prácticas mixtas, sin estandarización formal entre unidades académicas
FLEX (Flexibilidad)	Grado de flexibilidad en los requerimientos	Nominal: existen lineamientos institucionales, pero cada área puede adaptar criterios y formularios
RESL (Resolución de arquitectura)	Nivel de definición técnica antes del desarrollo	Bajo: la arquitectura funcional está en revisión, con herramientas diversas y sin integración consolidada
TEAM (Cohesión del equipo)	Nivel de comunicación y coordinación	Nominal: equipo técnico con experiencia parcial y coordinación inter área aún en consolidación
PMAT (Madurez del proceso)	Nivel de formalidad en gestión de proyectos	Bajo: no se aplican metodologías estandarizadas ni protocolos unificados de seguimiento técnico

Tabla de valores estándar

Nivel	Valor numérico
Muy bajo	6.20
Bajo	4.05
Nominal	3.00
Alto	2.00
Muy alto	1.00
Extra alto	0.00

Ejemplo

Supongamos que se asignan los siguientes niveles:

Factor	Nivel asignado	Valor	Justificación
PREC (Precedencia)	Bajo	4.05	El proceso de admisión combina prácticas digitales y presenciales, con escasa estandarización previa.
FLEX (Flexibilidad)	Nominal	3.00	Los requerimientos están guiados por criterios institucionales, pero cada área puede adaptar sus procedimientos.
RESL (Resolución de arquitectura)	Bajo	4.05	La arquitectura técnica aún está en revisión, con herramientas diversas en uso y sin integración consolidada.
TEAM (Cohesión del equipo)	Nominal	3.00	El equipo técnico tiene experiencia parcial y la coordinación entre áreas evaluadoras es intermitente.
PMAT (Madurez del proceso)	Bajo	4.05	No se aplican metodologías estandarizadas de gestión de proyectos ni protocolos unificados de seguimiento técnico.

$$\text{Suma total} = 4.05 + 3.00 + 4.05 + 3.00 + 4.05 = 18.15$$

$$B = 0.91 + 0.01 \times 18.15 = 1.0915 \approx 1.09$$

5.3.2 ¿Cómo se determina el EAF?

El EAF se obtiene como el **producto de 17 multiplicadores individuales**, cada uno asociado a un atributo del proyecto. Estos atributos están agrupados en categorías como:

1. Atributos del producto

- **Fiabilidad requerida (RELY):** ¿Qué tan crítico es que el sistema funcione sin fallos?
- **Tamaño de la base de datos (DATA):** ¿Qué volumen de datos debe manejar el sistema?
- **Complejidad del producto (CPLX):** ¿Qué tan complejas son las operaciones, algoritmos o reglas?

2. Atributos del personal

- Capacidad del equipo de análisis (ACAP)
- Capacidad del equipo de programación (PCAP)
- Experiencia en la aplicación (AEXP)
- Experiencia en la plataforma (PEXP)
- Experiencia en lenguaje y herramientas (LTEX)

3. Atributos del proyecto

- **Uso de herramientas de software (TOOL):** ¿Se utilizan herramientas modernas de desarrollo?
- **Restricciones de tiempo (SCED):** ¿Hay presión por cumplir plazos ajustados?
- **Multisitio (SITE):** ¿El equipo está distribuido geográficamente?
- **Métodos de desarrollo (PMAT):** ¿Qué tan formal es el proceso de gestión?

4. Atributos del hardware

- Tiempo de respuesta requerido (RUSE)
- Restricciones de ejecución (TIME)
- Memoria disponible (STOR)
- Volatilidad de la plataforma (PVOL)

5.3.2.1 ¿Cómo se determina el multiplicador?

◆ Paso 1: Evaluar el atributo

Se analiza el contexto del proyecto y se asigna un **nivel cualitativo** al atributo. Por ejemplo:

- ¿Qué tan crítica es la fiabilidad del sistema? → *Alto*
- ¿Qué volumen de datos se espera manejar? → *Nominal*
- ¿Qué tan compleja es la lógica del sistema? → *Muy alto*
- ¿Qué nivel de experiencia tienen los analistas? → *Bajo*
- ¿Qué herramientas de desarrollo se utilizan? → *Nominal*

◆ Paso 2: Consultar la tabla de multiplicadores

Cada nivel cualitativo tiene un **valor numérico** asociado. Aquí te muestro un ejemplo parcial de la tabla para algunos atributos:

Atributo	Muy bajo	Bajo	Nominal	Alto	Muy alto	Extra alto
RELY Fiabilidad requerida	0.75	0.88	1.00	1.15	1.40	—
DATA Tamaño de datos	—	0.94	1.00	1.08	1.16	—
CPLX Complejidad del producto	0.70	0.85	1.00	1.15	1.30	1.65
RUSE Reusabilidad requerida	—	0.95	1.00	1.07	1.15	1.24
DOCU Necesidad de documentación	0.89	0.95	1.00	1.06	1.13	—
TIME Restricciones de tiempo	—	—	1.00	1.11	1.29	1.63
STOR Restricciones de memoria	—	—	1.00	1.05	1.17	1.46
PVOL Volatilidad de la plataforma	—	0.87	1.00	1.15	1.30	—
ACAP Capacidad de analistas	1.46	1.19	1.00	0.86	0.71	—
PCAP Capacidad de programadores	1.42	1.17	1.00	0.86	0.70	—
PCON Continuidad del equipo	1.29	1.12	1.00	0.90	0.81	—
APEX Experiencia en la aplicación	1.22	1.10	1.00	0.88	0.81	—
PLEX Experiencia en la plataforma	1.19	1.09	1.00	0.91	0.85	—
LTEX Experiencia en lenguaje y herramientas	1.20	1.09	1.00	0.91	0.84	—
TOOL Uso de herramientas de desarrollo	1.17	1.09	1.00	0.90	0.78	0.65
SITE Distribución geográfica del equipo	1.22	1.09	1.00	0.93	0.86	0.80
SCED Restricciones de cronograma	1.43	1.14	1.00	1.00	—	—

Notas:

- Un guión (—) indica que ese nivel no es aplicable para ese atributo.
- Los valores fueron definidos empíricamente por el equipo de Barry Boehm y pueden ajustarse según el contexto institucional.

5.3.2.2 Ejemplo

Supongamos que estás estimando el sistema de gestión de admisiones y asignás los siguientes valores a los atributos según la escala estándar:

Ejemplo parcial:

Atributo	Nivel asignado	Multiplicador	Justificación
RELY (fiabilidad)	Alto	1.10	El sistema debe garantizar trazabilidad y consistencia en el proceso de admisión, sin pérdidas de datos.
DATA (volumen de datos)	Nominal	1.00	El volumen de datos por expediente es moderado, pero constante.
CPLX (complejidad del producto)	Muy alto	1.30	El sistema debe coordinar múltiples etapas, actores y condiciones de revisión.
ACAP (capacidad de los analistas)	Bajo	1.10	El equipo técnico tiene experiencia parcial y limitada en integración institucional.
TOOL (uso de herramientas)	Nominal	1.00	Se utilizan herramientas estándar, sin automatización avanzada ni integración completa.

$$\text{EAF parcial} = 1.10 \times 1.00 \times 1.30 \times 1.10 \times 1.00 = 1.573$$

Luego se continúa con los 12 atributos restantes. El **EAF total** suele oscilar entre:

- **0.7** → Proyecto muy favorable (equipo experto, requisitos estables, herramientas avanzadas)
- **1.5 o más** → Proyecto desafiante (alta complejidad, baja cohesión, herramientas limitadas)

5.4. Ejemplo aplicado

Caso: Sistema de gestión de admisiones – Universidad Nacional de Las Colinas Supongamos que el sistema tiene un tamaño estimado de **25 KLOC**, y se identifican los factores de escala y atributos de esfuerzo relevantes para el entorno universitario.

5.4.1. Cálculo del Factor de Escala B – Ejemplo aplicado a Las Colinas

Fórmula

$$B = 0.91 + 0.01 \times \Sigma(\text{SFi})$$

Donde SFi son los valores numéricos asignados a los cinco factores de escala.

Paso 1: Evaluar los 5 factores de escala

Factor de escala	Descripción	Nivel asignado	Valor numérico
PREC (Precedencia)	Nivel de madurez del proceso	Bajo	4.05
FLEX (Flexibilidad)	Grado de adaptación de requerimientos	Nominal	3.00
RESL (Resolución de arquitectura)	Nivel de definición técnica	Bajo	4.05
TEAM (Cohesión del equipo)	Comunicación y coordinación	Nominal	3.00
PMAT (Madurez del proceso)	Formalidad en gestión de proyectos	Bajo	4.05

Justificación contextual: El sistema de admisión en Las Colinas presenta una arquitectura híbrida aún en evolución, con prácticas informales entre áreas evaluadoras, requerimientos parcialmente definidos y un equipo técnico con experiencia limitada en integración institucional.

Paso 2: Sumar los valores

$$\Sigma(SFi) = 4.05 + 3.00 + 4.05 + 3.00 + 4.05 = \mathbf{18.15}$$

Paso 3: Calcular B

$$\mathbf{B = 0.91 + 0.01 \times 18.15 = 1.0915 \approx 1.09}$$

En algunos casos, puede redondearse a **1.10 o 1.12** si se detectan factores adicionales como alta sensibilidad institucional, necesidad de trazabilidad o impacto en procesos críticos.

5.4.2. Estimación de esfuerzo

En el caso de Las Colinas, se identificaron condiciones que afectan el esfuerzo:

- **Alta complejidad funcional** (por coordinación entre áreas, validación documental, trazabilidad del expediente).
- **Alta fiabilidad requerida** (el sistema debe garantizar consistencia y evitar pérdidas de información).
- **Baja experiencia del equipo** (equipo técnico con trayectoria parcial en sistemas administrativos integrados).
- **Variabilidad operativa** (diferencias entre unidades, estilos de trabajo y herramientas utilizadas).

Atributos más influyentes

Atributo	Nivel asignado	Multiplicador
RELY (fiabilidad requerida)	Alto	1.15
CPLX (complejidad del producto)	Muy alto	1.30
ACAP (capacidad de analistas)	Bajo	1.19
PCAP (capacidad de programadores)	Bajo	1.17
TOOL (uso de herramientas)	Nominal	1.00
SITE (equipo distribuido)	Nominal	1.00
TIME (restricciones de tiempo)	Nominal	1.00
PVOL (volatilidad de la plataforma)	Alto	1.15
STOR (restricciones de memoria)	Nominal	1.00

$$\mathbf{EAF\ parcial = 1.15 \times 1.30 \times 1.19 \times 1.17 \times 1.00 \times 1.00 \times 1.00 \times 1.15 \times 1.00 \approx 1.25}$$

5.4.3. Aplicando la fórmula

$$\mathbf{Esfuerzo\ estimado = 2.94 \times (25)^{1.12} \times 1.25 \approx 122\ persona-mes}$$

Esto significa que, por ejemplo, si el equipo está conformado por **5 personas**, se necesitarían aproximadamente **24 meses** para completar el proyecto, **si no se aplican estrategias de aceleración, modularización o automatización.**

💰 6. Estimación de costos por perfil técnico

Una vez estimado el esfuerzo total en persona-mes, es posible traducirlo en términos monetarios considerando la cantidad de integrantes del equipo y los roles específicos que intervienen en el desarrollo del sistema. Esta desagregación permite construir un presupuesto más realista y justificable, diferenciando funciones técnicas, niveles de especialización y momentos de participación en el ciclo de vida del proyecto.

En lugar de asumir un costo uniforme por persona, se propone distribuir el esfuerzo entre perfiles como analistas funcionales, desarrolladores backend y frontend, diseñadores UX/UI, testers, coordinadores técnicos y personal de soporte. Cada uno de estos roles tiene una carga de trabajo estimada y un valor referencial mensual, lo que permite calcular el costo total del proyecto con mayor precisión.

Esta estrategia no solo mejora la trazabilidad presupuestaria, sino que también facilita la planificación operativa, la asignación de recursos y la presentación de informes ante instituciones, financiadores o equipos de gestión.

🌱 6.1. ¿Por qué diferenciar roles?

- No todos los miembros del equipo tienen el mismo nivel de especialización ni el mismo costo por hora o mes.
- Algunos roles son más intensivos en ciertas etapas (por ejemplo, análisis al inicio, testing al final).
- Permite justificar presupuestos ante instituciones o financiadores con mayor transparencia.
- Facilita la planificación de contrataciones, capacitaciones o tercerizaciones.

👥 6.2. Roles típicos en proyectos de software

Rol	Función principal	Participación estimada	Costo relativo
Analista funcional	Relevamiento, modelado, validación de requerimientos	Alta en etapas iniciales	Medio/Alto
Diseñador UX/UI	Prototipos, interfaces, experiencia de usuario	Media en diseño	Medio
Desarrollador backend	Lógica del sistema, base de datos, APIs	Alta en desarrollo	Alto
Desarrollador frontend	Interfaces, interacción, validación en cliente	Alta en desarrollo	Medio/Alto
Tester / QA	Pruebas, validación, control de calidad	Alta en cierre	Medio
Coordinador técnico	Supervisión, integración, gestión operativa	Constante	Alto
Soporte / Documentador	Manuales, capacitación, asistencia operativa	Media en cierre	Bajo/Medio

6.3. Ejemplo aplicado: Sistema de gestión de admisiones – Universidad Nacional de Las Colinas

Supongamos que el esfuerzo total estimado para desarrollar el sistema de admisión es de **122 personas-mes**. Podés distribuirlo así:

Rol	% del esfuerzo	Personas-mes	Costo mensual estimado (ARS)	Costo total (ARS)
Analista funcional	15%	18.3	600.000	10.980.000
Desarrollador backend	25%	30.5	700.000	21.350.000
Desarrollador frontend	20%	24.4	650.000	15.860.000
Tester / QA	10%	12.2	500.000	6.100.000
Coordinador técnico	10%	12.2	750.000	9.150.000
Soporte / Documentador	10%	12.2	450.000	5.490.000
UX/UI	10%	12.2	550.000	6.710.000

 **Costo total estimado ≈ ARS 75.640.000**

Los valores son referenciales y pueden ajustarse según honorarios institucionales, modalidad de contratación o convenios universitarios.

7 Estimación por analogía

La estimación por analogía es una técnica que permite proyectar el esfuerzo, tiempo o costo de desarrollo de un sistema basándose en la experiencia previa con proyectos similares. En lugar de descomponer el sistema en componentes detallados, se parte de una visión global y se busca un “pariente funcional” que sirva como referencia.

¿Cómo funciona?

El analista identifica un sistema previamente desarrollado que comparta características clave con el nuevo proyecto: tipo de usuarios, complejidad funcional, entorno operativo, tecnología utilizada, etc. A partir de esa comparación, se ajustan los valores estimados según las diferencias detectadas (por ejemplo, mayor cantidad de usuarios, menor conectividad, más módulos, etc.).

Ejemplo aplicado

Si en años anteriores se desarrolló un sistema de inscripción para carreras de posgrado, ese proyecto puede servir como base para estimar el esfuerzo de un sistema de admisión general, ajustando por volumen de aspirantes, cantidad de etapas, y diversidad de documentación requerida.

Ventajas

- **Agilidad:** permite obtener estimaciones rápidas sin necesidad de modelado detallado.
- **Simplicidad:** ideal para presentaciones preliminares ante tomadores de decisión.
- **Utilidad estratégica:** ayuda a validar la viabilidad general antes de invertir en análisis profundo.

Limitaciones

- **Dependencia de antecedentes confiables:** si el proyecto de referencia no fue bien documentado, la analogía puede inducir errores.
- **Sensibilidad a diferencias contextuales:** cambios en tecnología, equipo, o entorno pueden invalidar la comparación.
- **Menor precisión:** no permite identificar riesgos específicos por módulo ni prever cuellos de botella técnicos.

Aplicación típica

- Proyectos pequeños o pilotos institucionales
- Etapas iniciales de planificación
- Presentaciones ante comités o autoridades para justificar la inversión inicial
- Comparación entre alternativas de solución

8. Estimación por componentes

La estimación por componentes es una técnica que permite calcular el esfuerzo y los recursos necesarios para desarrollar un sistema a partir de su descomposición funcional. En lugar de tomar el sistema como una unidad global, se lo divide en **módulos operativos**, cada uno con sus propias entradas, salidas, reglas de negocio y condiciones de uso.

¿Cómo funciona?

El analista identifica los principales bloques funcionales del sistema (por ejemplo, “registro de aspirantes”, “validación documental”, “asignación de etapas”) y estima el esfuerzo requerido para desarrollar cada uno. Esta estimación puede basarse en puntos de función, líneas de código, horas de trabajo o experiencia previa con módulos similares.

Ejemplo aplicado

En el sistema de admisión de Las Colinas, se podrían definir componentes como:

- Módulo de carga de datos personales
- Módulo de revisión documental
- Módulo de asignación interárea
- Módulo de generación de informes Cada uno se estima por separado, permitiendo ajustar el esfuerzo según su complejidad, criticidad y grado de automatización.

Ventajas

- **Flexibilidad:** permite ajustar estimaciones por módulo según prioridades o restricciones.
- **Planificación progresiva:** facilita la implementación por etapas y la asignación de equipos especializados.
- **Claridad operativa:** ayuda a visualizar el sistema como conjunto de funciones concretas, útiles para el diseño técnico y pedagógico.

Limitaciones

- **Requiere definición precisa:** si los módulos no están bien delimitados, la estimación puede ser inconsistente.
- **Demanda más tiempo inicial:** exige trabajo previo de análisis funcional y arquitectura.
- **Sensibilidad a cambios:** si se modifican los requerimientos, puede ser necesario recalcular varios componentes.

Aplicación típica

- Sistemas con arquitectura distribuida o modular
- Plataformas universitarias con múltiples actores y etapas
- Aplicaciones escalables que requieren mantenimiento y evolución futura
- Proyectos donde se busca justificar el esfuerzo por función o por área

9.- Resumen

Esta guía ha recorrido los elementos fundamentales para abordar el diseño y la planificación de sistemas funcionales en contextos comunitarios o institucionales. A partir de la definición de objetivos específicos, se establecieron criterios para derivar requerimientos funcionales, construir módulos operativos y asegurar la trazabilidad entre componentes.

El estudio de factibilidad permitió evaluar la viabilidad técnica, organizacional y económica del sistema, integrando factores como la complejidad esperada, la experiencia del equipo y las condiciones del entorno. Este análisis no solo orienta decisiones estratégicas, sino que también fortalece la transparencia y la justificación de los recursos involucrados.

Se aplicaron métodos de estimación que permiten proyectar el esfuerzo y los costos de desarrollo con base en parámetros observables y ajustables. En el caso del sistema de admisión de la Universidad Nacional de Las Colinas:

- Se estimó un tamaño funcional de **112 puntos de función**, equivalente a **25 KLOC**, lo que permitió calcular un esfuerzo total de aproximadamente **122 persona-mes** utilizando el modelo **COCOMO II** con un exponente B de **1.09** y un factor de ajuste EAF de **1.25**.
- La distribución del esfuerzo por rol técnico permitió proyectar un costo total estimado de **ARS 75.640.000**, considerando perfiles como analista funcional, desarrolladores, tester, coordinador técnico, soporte y UX/UI.

Finalmente, se presentaron distintos métodos de estimación de costos, incluyendo el enfoque por líneas de código (LOC), puntos de función (PF) y el modelo COCOMO II. También se compararon enfoques como la **estimación por analogía** () , útil en etapas tempranas, y la **estimación por componentes** () , ideal para sistemas modulares como el propuesto.

Esta guía no busca ofrecer respuestas cerradas, sino habilitar procesos de reflexión, simulación y mejora continua. Cada herramienta presentada puede adaptarse a distintos escenarios, y su valor reside en la capacidad de empoderar a los equipos para tomar decisiones informadas, construir soluciones sostenibles y fortalecer la calidad operativa de los sistemas que diseñan.

Bibliografía

- Sommerville, I. (2011). *Software engineering* (9th ed.). Pearson Education.
- Yourdon, E. (1989). *Modern structured analysis*. Prentice Hall.
- Pressman, R. S. (2010). *Software engineering: A practitioner's approach* (7th ed.). McGraw-Hill.
- IEEE. (1998). *IEEE Standard for Software Requirements Specifications* (IEEE Std 830-1998). Institute of Electrical and Electronics Engineers.
- Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., ... & Steece, B. (2000). *Software cost estimation with COCOMO II*. Prentice Hall.
- International Function Point Users Group (IFPUG). (2010). *Function Point Counting Practices Manual* (Release 4.3.1). IFPUG.